Cancellation coverage matrix — operation-scoped cancellation (`45cc661`)

Candidate	**`45cc661d2898e52ad8512083929535d1667050d7`** on `fix/operation-scoped-cancellation`
Lineage	`f764c95` (B1 recovery scope + B6 killable subprocess) → `b317833` (B2/B3/B7 production-boundary tests + B7 test seam) → `0d05556` (recovery window = real remaining fetch-phase budget) → `782f99d` (bounded in-flight browser interruption) → `e4894ae` (obsolete-work settlement for coalesced callers) → `45cc661` (real-IPC proc-admission withdrawal test + explicit unattempted-URL outcomes)
Base (production)	`origin/main` = `8c03b07` (untouched)
Purpose	Honest requirement→test→assertion map for every cancellation path, stating for each case **what level of realism** the test exercises (real Chromium child / real isolated subprocess / real OS fork+IPC / real Postgres / broker unit / in-process unit) and **what it does not prove**.

This document answers reviewer point 2. It is deliberately conservative: where a test proves *ordering* or *admission-gate* behaviour rather than *mid-flight interruption*, it says so, and the open gaps are listed explicitly in the final section rather than folded into a “covered” row.

Realism legend

Level	Meaning
R — real Chromium child	A real isolated job child (`runPipelineIsolated` → `job-child.ts`) drives the **production** browser call sites against a local HTTP server, with a real Chromium process observed via `/proc`.
R(page) — real Chromium page	A real Chromium page (`PUPPETEER_EXECUTABLE_PATH`) is held mid-operation and interrupted; wall-clock bound asserted against the production interrupt helper.
S — real isolated subprocess	A real OS child process (pdf-parse worker via `runProcessCollect` / `extractViaPdfParseSubprocess`) is spawned and killed; the test observes real SIGKILL / reaping.
I — real fork + IPC	A real OS child process is `fork` ed and driven over the production IPC channel (`job-child.ts` → `isolation-ipc.ts` → parent broker); admission withdrawal observed across the process boundary.
D — real Postgres	The assertion is made against the real `documents` / `companies` tables in the integration Postgres (`ciq-int-pg`), through the production storage functions.
U(broker)	The real admission broker / in-flight map / scope machinery is exercised in-process (no network, no Chromium), driving the actual production functions (`acquireProcSlot` , `processDocument` , `runCompanyPdfRecovery`).
U(unit)	Pure in-process unit assertion over the scope/deadline primitives.

“Production code path” means the assertion runs through the same function production calls — not a re-implementation in the test.

1. Deadline / scope propagation (B1 — company PDF recovery)

Boundary: `runCompanyPdfRecovery` (pipeline.ts), called in production (default `fetchFn` = real `fetchIssuerPdfsWithPrimedSession`).

#	Requirement	Test :: subtest	Key assertion	Level
1.1	One scope is armed and **passed to every fetch call** across all origins	`company-pdf-recovery-boundary.test.ts` :: B1-1	identical `scope` object observed at the fetch boundary for every origin	U(broker)
1.2	Recovery budget is **capped by `min(PDF_RECOVERY_MAX_BUDGET_MS, remainingMs)`**	B1-2	armed deadline ≈ `remainingMs` when `remainingMs < cap`	U(unit)
1.3	**No fetch is dispatched after the deadline**; the untried tail stays retryable	B1-3	origins after expiry are never passed to `fetchFn`; tail URLs remain unattempted (retryable)	U(broker)
1.4	Attempt budget caps total attempts **across** origins (company-wide, not per-origin)	B1-4	total fetch attempts ≤ attemptBudget summed across origins	U(broker)
1.5	Healthy path: all origins fetched, nothing filtered	B1-5	every origin attempted; no spurious abort	U(broker)
1.6	Default `fetchFn` routes through the **real admission boundary** and withdraws at the company deadline	B1-6	with the production default `fetchFn`, a queued waiter withdraws when the company deadline elapses	U(broker) — real `acquireProcSlot`
1.7	**Production recovery window = the real remaining fetch-phase budget**	B1-7	`remainingFetchBudgetMs(fetchPhaseStart)` returns the budget **minus elapsed**, clamped to a	U(unit) — real `remainingFetchBudgetMs`

#	Requirement	Test :: subtest	Key assertion	Level
	(not the 600 000 ms constant)		floor; never the raw constant	

Mutation proof (B1-1): removing scope propagation (`fetchFn(origin, slice, { outcomes, scope })` → `{ outcomes }`) makes **B1-1 fail** (`AssertionError: a scope was passed to the fetch boundary (was DEAD/omitted before B1)`); B1-2/ B1-3 cascade. Reverted clean.

Mutation proof (B1-7, this session): replacing the body of `remainingFetchBudgetMs` with `return budgetMs;` (ignore elapsed — `MUTATION_MARKER_B2`) makes **B1-7 fail** (`6 pass / 1 fail` : the window no longer shrinks with elapsed time). Reverted clean; `grep MUTATION_MARKER` empty, tree matches HEAD.

What B1 does NOT prove: B1 operates at the recovery-orchestration boundary, not against a live network fetch. It proves the scope is *constructed, capped, and passed*, the window tracks real elapsed budget, and dispatch stops after the deadline — it does not prove that an HTTP fetch already in flight is torn down mid-transfer (that is the active-interruption boundary, §6 and §7).

---

2. Process-admission cancellation (B2 — `acquireProcSlot` , in-process)

Boundary: `acquireProcSlot` local-broker queue (processor.ts), queued-waiter abort listener.

#	Requirement	Test :: subtest	Key assertion	Level
2.1	A **queued** proc waiter whose scope expires **withdraws** and is never granted a slot	`cancellable-proc-admission-bound-ary.test.ts` :: B2-1	waiter settles `withdrawn`, not `hung`; no slot consumed	U(broker)
2.2	The next waiter is admitted promptly once the holder re-leases	B2-2	next waiter granted; `active` returns to expected	U(broker)
2.3	Grant/abort race (abort **before** release): waiter withdraws, slot returns cleanly — no leak, no phantom grant	B2-3	`active` drains to 0; no double-grant	U(broker)
2.4	A **pre-aborted** scope is refused at admission and consumes no slot	B2-4	throws `OperationAbortedError`; `active` unchanged	U(broker)

Mutation proof (B2): removing the queued-waiter abort listener makes **B2-1 fail** (`'hung' !== 'settled'`), and B2-2/B2-3/B2-4 cascade with `proc slot leaked (active=1)`. Reverted clean.

2b. Process-admission withdrawal across a real process boundary (real fork + IPC)

Boundary: a real isolated child (`runPipelineIsolated` → `job-child.ts`) whose test module calls the **production** `processorSlotBroker.proc.acquire(scope)` → `acquireProcSlot` → `remoteAcquire("proc", …)` over the real IPC channel to the parent broker.

#	Requirement	Test :: subtest	Key assertion	Level
2b.1	A child blocked at **proc admission** whose scope aborts **withdraws across IPC** and surfaces an `OperationAbortedError` tagged `proc_admission`	`isolated-proc-admission-bound-ary.test.ts`	child reports `isOperationAborted === true` with reason `proc_admission`, promptly after abort (≪ grant would arrive)	**I**

Mutation proof (2b, this session): in `acquireProcSlot`, breaking the abort-error **type** fed to `remoteAcquire` (`() => new OperationAbortedError(…, "proc_admission")` → `() => new Error("MUTATION_MARKER_B5 generic …")`) makes the test **fail fast** (child still withdraws at ~700 ms but surfaces `isOperationAborted: false`). Reverted clean. (A weaker mutation — suppressing the `cancelAcquire` IPC message — did **not** fail the test, because the child exits ~50 ms after `run()` returns and the parent reclaims the queued request on child exit; that masking is documented here rather than hidden, and is why the *error-type* mutation is the load-bearing one.)

This case is strictly stronger in realism than §2's in-process U(broker): the admission queue, the abort, and the withdrawal all cross a real OS process boundary over the production IPC protocol.

---

3. Stalled-extraction cancellation (B3 — `runProcessCollect` subprocess)

Boundary: `runProcessCollect` abort registration, SIGKILL in `onAbort`.

#	Requirement	Test :: subtest	Key assertion	Level
3.1	A **hung extraction child** is SIGKILLed on scope abort; parent stays responsive; slot restored	`stalled-extraction-bound-ary.test.ts` :: B3	parent settles **promptly after abort** (elapsed ≪ child timeout); slot restored	**S**
3.2	`extractViaPdfParseSubprocess` throws **retryable** on an already-aborted scope and takes **no** slot	B3/B2	throws; `active` unchanged	**S**

Mutation proof (B3): disabling the abort registration block makes **B3 fail** with `parent settled promptly after abort (elapsed=60005ms)` — the child runs the **full 60 s timeout** instead of being killed at abort. Reverted clean. This is the clearest active-interruption proof in the subprocess layer: a real child process is really killed at the abort instant.

---

4. In-flight coalescing cancellation + obsolete-work settlement (B7 — `processDocument`)

Boundary: `processDocument` in-flight dedup + liveness guard + coalesced-caller self-settlement (processor.ts).

#	Requirement	Test :: subtest	Key assertion	Level
4.1	Concurrent same-URL callers **coalesce to one** underlying fetch	`inflight-coalescing-boundary.test.ts` :: B7-1	exactly 1 underlying fetch for 2 concurrent callers	U(broker)
4.2	A new caller does **NOT** coalesce onto an in-flight entry whose scope has **aborted** — it starts its own attempt	B7-2	2 underlying fetches (aborted entry not reused)	U(broker)
4.3	A caller **coalesced onto** an in-flight op whose **own** scope then aborts **settles to a terminal state** (does not hang waiting on the shared op it no longer cares about)	B7-3	the coalesced caller's promise settles on its own abort via `Promise.race([existing.promise, callerScope.whenAborted("coalesced_wait")])`; it never waits past its own deadline	U(broker)

Mutation proof (B7), three mutants:
- Disable coalescing (`if (false && existing …)`) → **B7-1 fails** (`2 !== 1`).
- Drop the liveness check (ignore `scope.aborted()`) → **B7-2 fails** (`1 !== 2` — the new caller wrongly coalesces onto the aborted entry).
- Drop the coalesced-caller race (`Promise.race([existing.promise, callerScope.whenAborted(…)])` →

`return existing.promise;` — `MUTATION_MARKER_B4`, this session) → **B7-3 fails** (`2 pass / 1 fail`: the coalesced caller hangs on the obsolete shared op past its own abort). Reverted clean.

All reverted clean. Three independent mutants are needed because the three guards (coalesce, liveness, self-settlement) are independently load-bearing.

---

5. Browser lifecycle close/reap ordering (real Chromium) — `browser-close-order.test.ts`

Boundary: real isolated job children driving **production** browser call sites (`fetchPdfViaBrowser`, `fetchIssuerPdfsWithPrimedSession`), `MAX_CONCURRENT_BROWSER=1`, live Chromium observed via `/proc` at every permit grant/return. Invariant asserted on **every** exit path: **work stopped → Chromium closed → termination confirmed → permit returned**.

#	Requirement	Test :: subtest	Key assertion	Level
5.1	**Normal op:** marker extracted; Chromium gone before the permit returns	success path	`r.ok`, marker matched, `liveChromium == []` at release	**R**
5.2	**Fetch failure** (connection reset; direct + pdfRecovery): close before release	fetch-failure path	no text yielded; `liveChromium == []` at both releases	**R**
5.3	**Cancellation — admission gates:** a caller refused **after** acquiring the permit, and one refused **before** acquiring, both raise `BrowserUnavailableError: lifecycle cancelled`; permits return with Chromium closed	cancellation path (B_postAcquire, C_preAcquire)	both error strings match; `liveChromium == []` at release	**R**
5.4	**Cancellation — in-flight:** a fetch already holding the permit at the cancel instant **completes**, then Chromium closes before the permit returns	cancellation path (A_inflight)	`A_inflight.ok == true`; `liveChromium == []` at release	**R**
5.5	**Child death:** pipeline deadline → SIGKILL while holding the permit → parent group	child-death path	job fails; `liveChromium == []` at release; broker drained	**R**

#	Requirement	Test :: subtest	Key assertion	Level
	reap before `releaseAll`			

Mutation proof (ordering): documented in the prior browser-lifetime review — releasing the permit *before* closing Chromium makes these tests fail (the `/proc` scan sees a live Chromium at release). Not re-run here (no code in that path changed in this candidate).

What §5 proves and what it does NOT:
- It **proves** the close/reap **ordering invariant** on all four exit paths with a **real Chromium child**, and it proves cancellation is honoured at the **admission gates** (post-acquire and pre-acquire refusal — 5.3).
- Row 5.4 is explicit: an in-flight fetch holding the permit **runs to completion** and cancellation at this layer takes effect at the **next gate**. The bounded mid-flight interruption of the active CDP op is a separate, newly-added path — §5b — and remains **partial** (see §6/§7).

5b. Bounded in-flight interruption of an active browser op (real Chromium page)

Boundary: `installBoundedPageInterrupt` (processor.ts), wired into the **`fetchPdfViaBrowser`** path. On scope abort it drives the active page toward settlement within `BROWSER_ABORT_GRACE_MS` (500 ms) plus a gate round-trip, rather than letting the in-flight CDP op run to its own timeout.

#	Requirement	Test :: subtest	Key assertion	Level
5b.1	A browser op **already in flight** when the scope aborts is driven to settle within the grace bound, not left to its own long timeout	`browser-inf-light-inter-rupt.test.ts`	the held page settles in **≪ the un-bounded wall time** once the scope aborts (bounded by `BROWSER_ABORT_GRACE_MS` + gate round-trip)	**R(page)**

Mutation proof (5b, this session): replacing the `installBoundedPageInterrupt` `onAbort` body with a no-op (`MUTATION_MARKER_B3 in-flight interrupt disabled`) makes the test **fail** — the page settles at **8169 ms** instead of inside the `< 3000 ms` bound. Reverted clean.

Scope of this fix (honest): the bounded interrupt is wired **only** into `fetchPdfViaBrowser`. The `fetchWithBrowser` HTML path is **not** wired to it — see §6.1 and §7. This row proves the mechanism works on the path it is installed on; it does not claim universal mid-flight interruption.

---

6. Honest gaps — requirements NOT fully proven by behavioural test

These are stated so the reviewer does not read a green suite as more than it is.

1. **Active in-page CDP interruption is bounded on one path only.** `installBoundedPageInterrupt` now bounds an in-flight CDP op to `BROWSER_ABORT_GRACE_MS` + a gate round-trip on the `fetchPdfViaBrowser` path (§5b, proven with a real Chromium page). The `fetchWithBrowser` **HTML path is still not wired** to this helper — an HTTP fetch / `page.goto` / `page.evaluate` / WAF challenge already executing on that path when the scope aborts is still not torn down mid-operation, and falls back to the next admission gate (B2/B7 refusal), dispatch suppression (B1-3), and ultimately the job-isolation SIGKILL (§5.5). This is a real, named remaining gap.
2. **Deadline propagation into the browser fetch is bounded, not threaded.** B1 proves the scope is *passed* to the fetch boundary and that *dispatch* stops after the deadline; it does not prove a per-request network/socket timeout equal to `remainingMs` is applied inside `fetchIssuerPdfsWithPrimedSession`. The effective ceiling on an already-started browser fetch on the un-wired path is the pipeline deadline → SIGKILL, not a scope-derived socket timeout.
3. **(RESOLVED in this candidate)** The production recovery window previously degenerated to the `PDF_RECOVERY_MAX_BUDGET_MS` (600000) constant because the call site omitted `remainingMs`. The call site now feeds `remainingFetchBudgetMs(fetchPhaseStart)` — the real budget minus elapsed, clamped to a floor — so the per-company remaining budget is honoured in production, not just in the test. Proven by B1-7 and its mutation (§1). The constant is unchanged; only how the window is *derived* changed.
4. **Specific production stalls remain unobserved.** The Idemitsu ~18-minute single-step and the AXA analysis-prep ~7m26s stall (see `TIMEOUT_INVESTIGATION.md`) are not reproduced by any test here; they are out of scope for this cancellation patch and are not claimed fixed.

---

7. Unattempted-URL outcomes on cancellation (explicit terminal state) — `unattempted-cancelled-boundary.test.ts`

Boundary: `markUnattemptedCancelled` (pipeline.ts), invoked at the fetch-loop cancel break. Before this candidate, the top-of-loop `if (cancelCheck?.()) break;` left still-`pending` documents silently in `pending` (the budget-exceeded path already force-marked its tail `budget_exceeded`, but the cancel path did not). The helper now walks the still-`pending` accepted documents and records each as fetch-dead with reason `not_attempted_cancelled`, giving every URL an explicit terminal outcome on cancellation.

#	Requirement	Test :: subtest	Key assertion	Level
7.1	On a cancel break, every still-**pending** accepted document for the company is recorded **fetch-dead** with reason `not_attempted_cancelled`	`unattempted-cancelled-bound-ary.test.ts` :: B6-1	all 3 seeded pending docs become `fetch_status='dead'`, `failure_reason='not_attempted_cancelled'` in real Postgres	**D**
7.2	Documents already in a terminal state are **not** overwritten; the helper only touches `pending`	B6-2	non-pending docs are left untouched	**D**

Mutation proof (7, this session): appending `&& false /* MUTATION_MARKER_B6 silent-drop */` to the pending filter in `markUnattemptedCancelled` makes **B6-1 fail** (`0 !== 3` — no doc marked; the silent-`pending` leak is back) while B6-2 still passes. Reverted clean.

Summary

Boundary	Behavioural test	Mutation-fail demo (this session)	Realism
B1 deadline/scope propagation + recovery window	7 subtests (B1-1..B1-7)	scope-drop → B1-1 fail; `return budgetMs` → B1-7 fail	U(broker)/U(unit)
B2 proc admission (in-process)	4 subtests (B2-1..B2-4)	listener-removed → B2-1 `hung`	U(broker)
B2b proc admission across real fork+IPC	1 subtest	generic-error type → withdrawal untagged, fail-fast	**I (real fork+IPC)**
B3 stalled extraction	2 subtests	abort-reg-disabled → full 60 s	**S (real kill)**
B6(sub) killable extraction	2 subtests	(subprocess boundary; see REVIEW_PACKAGE)	**S (real subprocess)**
B7 in-flight coalescing + settlement	3 subtests (B7-1..B7-3)	3 mutants → B7-1 / B7-2 / B7-3 fail	U(broker)
Browser close/reap ordering	4 paths	release-first → ordering fail (prior)	**R (real Chromium)**
Bounded in-flight page interrupt (PDF path)	1 subtest	onAbort no-op → 8169 ms vs < 3000 ms	**R(page) (real Chromium)**
Unattempted-URL terminal outcomes	2 subtests (B6-1, B6-2)	silent-drop → B6-1 `0 !== 3`	**D (real Postgres)**

11 test files, all green on `45cc661`: 33 `node:test` subtests across 10 files (operation-scoped-cancellation 7, company-pdf-recovery-boundary 7, cancellable-proc-admission-boundary 4, inflight-coalescing-boundary 3, stalled-extraction-boundary 2, pdf-parse-subprocess-equivalence 2, browser-close-order 4, browser-inflight-interrupt 1, isolated-proc-admission-boundary 1, unattempted-cancelled-boundary 2) + `scoring-cancellation.unit` (60 passed, 0 failed; custom runner). 0 failures anywhere.

Every mutation in this document was applied, observed to fail the named assertion, and reverted to a clean tree **in this session** (`grep -rn MUTATION_MARKER server/` → none; `git diff --quiet HEAD -- server/lib/processor.ts server/lib/pipeline.ts` → clean). The gaps in §6 are real and are the honest boundary of what the suite proves.